28-新特性：Tomcat和Jetty如何处理SpringBoot应用-

为了方便开发和部署，Spring Boot在内部启动了一个嵌入式的Web容器。我们知道Tomcat和Jetty是组件化的设计，要启动Tomcat或者Jetty其实就是启动这些组件。在Tomcat独立部署的模式下，我们通过startup脚本来启动Tomcat，Tomcat中的Bootstrap和Catalina会负责初始化类加载器，并解析`server.xml`和启动这些组件。

在内嵌式的模式下，Bootstrap和Catalina的工作就由Spring Boot来做了，Spring Boot调用了Tomcat和Jetty的API来启动这些组件。那Spring Boot具体是怎么做的呢？而作为程序员，我们如何向SpringBoot中的Tomcat注册Servlet或者Filter呢？我们又如何定制内嵌式的Tomcat？今天我们就来聊聊这些话题。

Spring Boot中Web容器相关的接口

既然要支持多种Web容器，Spring Boot对内嵌式Web容器进行了抽象，定义了**WebServer**接口：

```
public interface WebServer {
    void start() throws WebServerException;
    void stop() throws WebServerException;
    int getPort();
}
```

各种Web容器比如Tomcat和Jetty需要去实现这个接口。

Spring Boot还定义了一个工厂**ServletWebServerFactory**来创建Web容器，返回的对象就是上面提到的WebServer。

```
public interface ServletWebServerFactory {
    WebServer getWebServer(ServletContextInitializer... initializers);
}
```

可以看到getWebServer有个参数，类型是**ServletContextInitializer**。它表示ServletContext的初始化器，用于ServletContext中的一些配置：

```
public interface ServletContextInitializer {
    void onStartup(ServletContext servletContext) throws ServletException;
}
```

这里请注意，上面提到的getWebServer方法会调用ServletContextInitializer的onStartup方法，也就是说如果你想在Servlet容器启动时做一些事情，比如注册你自己的Servlet，可以实现一个ServletContextInitializer，在Web容器启动时，Spring Boot会把所有实现了ServletContextInitializer接口的类收集起来，统一调它们的onStartup方法。

仅供个人学习 请勿传播 用户标记 mannix小童鞋

为了支持对内嵌式Web容器的定制化，Spring Boot还定义了**WebServerFactoryCustomizerBeanPostProcessor**接口，它是一个BeanPostProcessor，它在postProcessBeforeInitialization过程中去寻找Spring容器中WebServerFactoryCustomizer

类型的Bean，并依次调用WebServerFactoryCustomizer

接口的customize方法做一些定制化。

```
public interface WebServerFactoryCustomizer<T extends WebServerFactory> {
    void customize(T factory);
}
```

内嵌式Web容器的创建和启动

铺垫了这些接口，我们再来看看Spring Boot是如何实例化和启动一个Web容器的。我们知道，Spring的核心是一个ApplicationContext，它的抽象实现类AbstractApplicationContext

实现了著名的**refresh**方法，它用来新建或者刷新一个ApplicationContext，在refresh方法中会调用onRefresh方法，AbstractApplicationContext的子类可以重写这个方法onRefresh方法，来实现特定Context的刷新逻辑，因此ServletWebServerApplicationContext就是通过重写onRefresh方法来创建内嵌式的Web容器，具体创建过程是这样的：

```
@Override
protected void onRefresh() {
     super.onRefresh();
     try {
        //重写onRefresh方法，调用createWebServer创建和启动Tomcat
        createWebServer();
     }
     catch (Throwable ex) {
     }
}

//createWebServer的具体实现
private void createWebServer() {
    //这里WebServer是Spring Boot抽象出来的接口，具体实现类就是不同的Web容器
    WebServer webServer = this.webServer;
    ServletContext servletContext = this.getServletContext();

    //如果Web容器还没创建
    if (webServer == null && servletContext == null) {
        //通过Web容器工厂来创建
        ServletWebServerFactory factory = this.getWebServerFactory();
        //注意传入了一个"SelfInitializer"
        this.webServer = factory.getWebServer(new ServletContextInitializer[]{this.getSelfInitializer()});

    } else if (servletContext != null) {
        try {
            this.getSelfInitializer().onStartup(servletContext);
        } catch (ServletException var4) {
          ...
        }
```

```
    }

    this.initPropertySources();
}
```

再来看看getWebSever具体做了什么，以Tomcat为例，主要调用Tomcat的API去创建各种组件：

```
public WebServer getWebServer(ServletContextInitializer... initializers) {
    //1.实例化一个Tomcat，可以理解为Server组件。
    Tomcat tomcat = new Tomcat();

    //2. 创建一个临时目录
    File baseDir = this.baseDirectory != null ? this.baseDirectory : this.createTempDir("tomcat");
    tomcat.setBaseDir(baseDir.getAbsolutePath());

    //3.初始化各种组件
    Connector connector = new Connector(this.protocol);
    tomcat.getService().addConnector(connector);
    this.customizeConnector(connector);
    tomcat.setConnector(connector);
    tomcat.getHost().setAutoDeploy(false);
    this.configureEngine(tomcat.getEngine());

    //4. 创建定制版的"Context"组件。
    this.prepareContext(tomcat.getHost(), initializers);
    return this.getTomcatWebServer(tomcat);
}
```

你可能好奇prepareContext方法是做什么的呢？这里的Context是指**Tomcat中的Context组件**，为了方便控制Context组件的行为，Spring Boot定义了自己的TomcatEmbeddedContext，它扩展了Tomcat的StandardContext：

```
class TomcatEmbeddedContext extends StandardContext {}
```

注册Servlet的三种方式

1. Servlet注解

在Spring Boot启动类上加上@ServletComponentScan注解后，使用@WebServlet、@WebFilter、@WebListener标记的Servlet、Filter、Listener就可以自动注册到Servlet容器中，无需其他代码，我们通过下面的代码示例来理解一下。

```
@SpringBootApplication
@ServletComponentScan
public class xxxApplication
{}
```

仅供个人学习 请勿传播 用户标记 mannix小查理

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {}
```

在Web应用的入口类上加上@ServletComponentScan， 并且在Servlet类上加上@WebServlet，这样SpringBoot会负责将Servlet注册到内嵌的Tomcat中。

2. ServletRegistrationBean

同时Spring Boot也提供了ServletRegistrationBean、 FilterRegistrationBean和ServletListenerRegistrationBean这三个类分别用来注册Servlet、Filter、Listener。假如要注册一个Servlet，可以这样做：

```
@Bean
public ServletRegistrationBean servletRegistrationBean() {
    return new ServletRegistrationBean(new HelloServlet(),"/hello");
}
```

这段代码实现的方法返回一个ServletRegistrationBean，并将它当作Bean注册到Spring中，因此你需要把这段代码放到Spring Boot自动扫描的目录中，或者放到@Configuration标识的类中。

3. 动态注册

你还可以创建一个类去实现前面提到的ServletContextInitializer接口，并把它注册为一个Bean，Spring Boot会负责调用这个接口的onStartup方法。

```
@Component
public class MyServletRegister implements ServletContextInitializer {

    @Override
    public void onStartup(ServletContext servletContext) {

        //Servlet 3.0规范新的API
        ServletRegistration myServlet = servletContext
                .addServlet("HelloServlet", HelloServlet.class);

        myServlet.addMapping("/hello");

        myServlet.setInitParameter("name", "Hello Servlet");
    }

}
```

这里请注意两点：

- ServletRegistrationBean其实也是通过ServletContextInitializer来实现的，它实现了ServletContextInitializer接口。
- 注意到onStartup方法的参数是我们熟悉的ServletContext，可以通过调用它的addServlet方法来动态注册新的Servlet，这是Servlet 3.0以后才有的功能。

Web容器的定制

我们再来考虑一个问题，那就是如何在Spring Boot中定制Web容器。在Spring Boot 2.0中，我们可以通过两种方式来定制Web容器。

第一种方式是通过通用的Web容器工厂ConfigurableServletWebServerFactory，来定制一些Web容器通用的参数：

```
@Component
public class MyGeneralCustomizer implements
  WebServerFactoryCustomizer<ConfigurableServletWebServerFactory> {

    public void customize(ConfigurableServletWebServerFactory factory) {
        factory.setPort(8081);
        factory.setContextPath("/hello");
     }
}
```

第二种方式是通过特定Web容器的工厂比如TomcatServletWebServerFactory来进一步定制。下面的例子里，我们给Tomcat增加一个Valve，这个Valve的功能是向请求头里添加traceid，用于分布式追踪。TraceValve的定义如下：

```
class TraceValve extends ValveBase {
    @Override
    public void invoke(Request request, Response response) throws IOException, ServletException {

        request.getCoyoteRequest().getMimeHeaders().
        addValue("traceid").setString("1234xxxxabcd");

        Valve next = getNext();
        if (null == next) {
            return;
        }

        next.invoke(request, response);
    }

}
```

跟第一种方式类似，再添加一个定制器，代码如下：

```
@Component
```

仅供个人学习 请勿传播 用户标记 mannix小食糙

```
public class MyTomcatCustomizer implements
        WebServerFactoryCustomizer<TomcatServletWebServerFactory> {

    @Override
    public void customize(TomcatServletWebServerFactory factory) {
        factory.setPort(8081);
        factory.setContextPath("/hello");
        factory.addEngineValves(new TraceValve() );

    }
}
```

本期精华

今天我们学习了Spring Boot如何利用Web容器的API来启动Web容器、如何向Web容器注册Servlet，以及如何定制化Web容器，除了给Web容器配置参数，还可以增加或者修改Web容器本身的组件。

课后思考

我在文章中提到，通过ServletContextInitializer接口可以向Web容器注册Servlet，那ServletContextInitializer跟Tomcat中的ServletContainerInitializer有什么区别和联系呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- 飞翔 2019-07-16 03:53:48
  老师 在digester 类里边的createStartDigester，方法，把enginer，host，context都实例化了，那wrapper是在哪里实例化的呢

- despacito 2019-07-13 22:24:33

老师，springboot 中 getWebServer方法的实现类不仅有tomcat，还有其他web容器，比如jetty，那为什么我们在运行启动类的时候默认都是用的tomcat容器，如果我运行启动类的时候想用jetty作为应用容器，应该怎么做？

作者回复2019-07-15 18:38:31
在pom文件中去掉对Tomcat的依赖，加入对jetty的依赖：

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<exclusions>
<exclusion>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
</exclusion>
</exclusions>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jetty</artifactId>
</dependency>

</dependencies>

- 刘冬 2019-07-13 13:37:37
和"飞翔"同问： 有@RestController，为什么还要自己去注册Servlet给Tomcat?
我感觉老师很善于将负责的问题、长的逻辑链讲的简洁清晰，还请老师帮忙详细说明一下。
谢谢！

作者回复2019-07-15 18:42:33
可能有些场景需要注册你自己写的一个Servlet提供辅助功能，与主程序分开

- 飞翔 2019-07-13 11:30:14
老师 sprongboot 不注册servlet 给tomcat 直接用@controller 就能实现servlet功能是咋回事呀

作者回复2019-07-15 18:24:17
springboot默认给我们注册了DispatcherSetvlet。

- 大漠落木 2019-07-13 09:28:11
@FunctionalInterface
org.springframework.boot.web.servlet.ServletContextInitializer
This interface is primarily designed to allow ServletContextInitializers to bemanaged by Spring and not the Servlet container.

仅供个人学习 请勿传播 用户标记 mannix小童鞋

javax.servlet.ServletContainerInitializer
ServletContainerInitializers (SCIs) are registered via an entry in the file META-INF/services/javax.servlet.ServletContainerInitializer that must beincluded in the JAR file that contains the SCI implementation.

- nightmare 2019-07-13 07:23:15
  感觉还是要跟着操作一下才能懂了